

16. Sicherheit

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2026

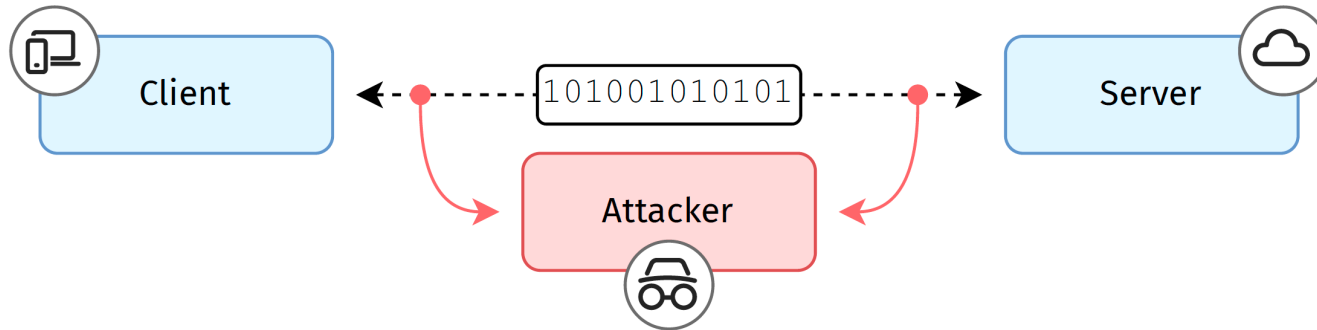
Vorschau

- Bedrohungen & Angriffsarten
- Fallbeispiel: ARP Spoofing - Probleme
- Sicherheitsmechanismen
- Symmetrische Verschlüsselung
- Asymmetrische Verschlüsselung
- Digitale Signatur
- Vergleich der Verschlüsselungsverfahren
- Fallbeispiel: ARP Spoofing - Lösungen

16.1 Bedrohungen & Angriffsarten

Grundlagen

- In einem verteilten System findet immer eine Kommunikation über ein Netzwerk statt.
- Ohne Gegenmaßnahmen kann sämtliche Kommunikation mitgelesen und/oder manipuliert werden



Wer kann angreifen?

- Eine angestellte Person im firmeninternen Netzwerk
- Der eigene Internetprovider, über welchen die gesamte Kommunikation läuft
- Eine Person, welche einen Fake-Hotspot (Wi-Fi) öffnet
- Ein Geheimdienst, welcher Knotenpunkte im Internet kontrolliert
- ...

Bedrohungen

- Engl. threats
- Unbefugter Zugriff auf Informationen und Dienste
- Beeinträchtigung der Leistungsfähigkeit eines Systems
- Unbefugtes Einbringen neuer Daten oder neuer „Dienste“, ...

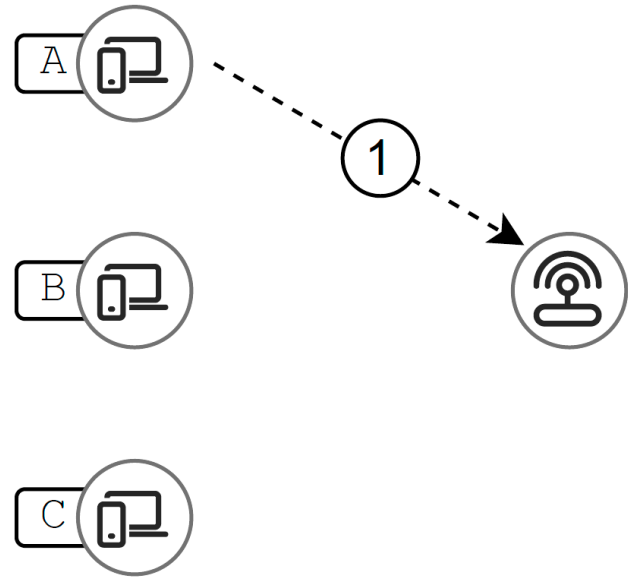
Angriffsarten

- **Denial of Service Attacke**: Versuch einen Dienst unbrauchbar zu machen
- **Lauschangriff**: unbefugtes Abhören einer Kommunikation
- **Maskerade**: Vorgeben einer falschen Identität
- **Infiltration**: eigene Komponenten werden eingeschleust
 - Virus: angehängt an legale Daten; wird unbeabsichtigt aktiviert; repliziert sich, ...
 - Wurm: unbefugtes Einbringen und Ausführen von Programmen (Sicherheitslöcher)
 - Trojanisches Pferd: angeblich legitime Programme entpuppen sich als Schädling.

16.2 Fallbeispiel: ARP Spoofing - Probleme

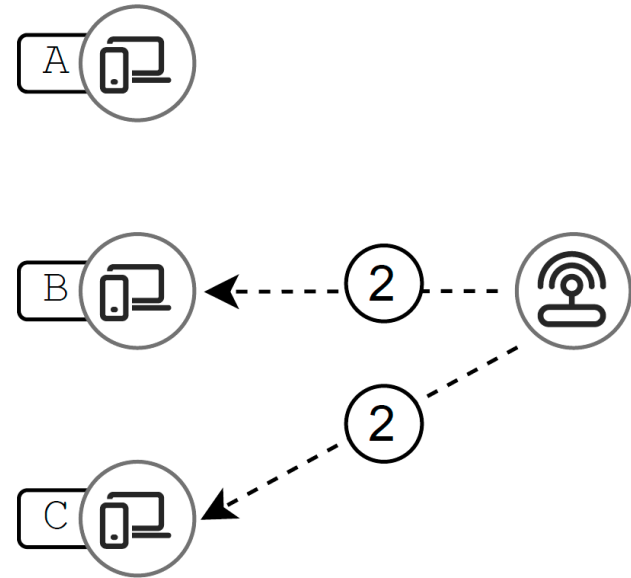
ARP-Protokoll

- Innerhalb eines Local Area Networks (LAN) wird das Address Resolution Protocols (ARP) verwendet, um zu einer IP-Adresse die zugehörige MAC-Adresse zu suchen.
- Ablauf:
 1. Host A schickt ARP-Nachricht an den Gateway (Router)



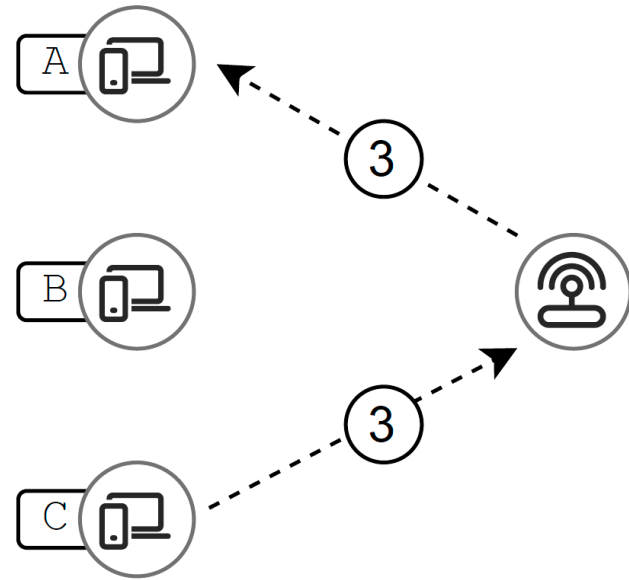
ARP-Protokoll

- Innerhalb eines Local Area Networks (LAN) wird das Address Resolution Protocols (ARP) verwendet, um zu einer IP-Adresse die zugehörige MAC-Adresse zu suchen.
- Ablauf:
 1. Host A schickt ARP-Nachricht an den Gateway (Router)
 2. Gateway leitet ARP-Nachricht an **alle** Hosts weiter



ARP-Protokoll

- Innerhalb eines Local Area Networks (LAN) wird das Address Resolution Protocols (ARP) verwendet, um zu einer IP-Adresse die zugehörige MAC-Adresse zu suchen.
- Ablauf:
 1. Host A schickt ARP-Nachricht an den Gateway (Router)
 2. Gateway leitet ARP-Nachricht an **alle** Hosts weiter
 3. Host C besitzt die richtige IP-Adresse und antwortet



- Eine Anfrage innerhalb des ARP-Protokolls beinhaltet unter anderem...
 - die IP-Adresse der Quelle
 - die MAC-Adresse der Quelle
 - die IP-Adresse des Ziels
- Erreicht die ARP-Anfrage das richtige Ziel (IP-Adresse stimmt überein), so antwortet das Ziel dem anfragenden Client mit seiner MAC-Adresse
⇒ MAC-Adresse des Ziels wird in der ARP-Tabelle des Clients gecached
- Da die Suche per Broadcast realisiert ist, wird das Ergebnis gecacht
⇒ Cache-Einträge sind je nach Konfiguration 1 – 2min. gültig

ARP Spoofing

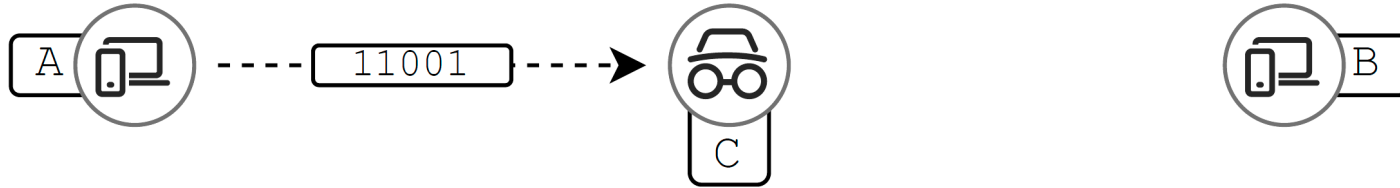
- Problem: Das ARP-Protokoll arbeitet ohne jegliche Authentifizierung
⇒ Nach einer Anfrage wird die erste darauf folgende Antwort genutzt, um die MAC-Adresse innerhalb der ARP-Tabelle zu aktualisieren
- Welche Konsequenzen sind damit verbunden?
- Jeder beliebige Computer im LAN kann falsche Antworten versenden!

ARP Spoofing

- Der Angreifer schickt kontinuierlich ARP-Antworten an das Angriffsziel und wartet, bis dieses die "falsche" MAC-Adresse eingetragen hat.
- Möglichkeiten nach einem erfolgreichen Angriff:
 - Anfragen verwerfen → Denial of Service
 - Anfragen mitlesen und/oder manipulieren → Man-in-the-Middle Attack

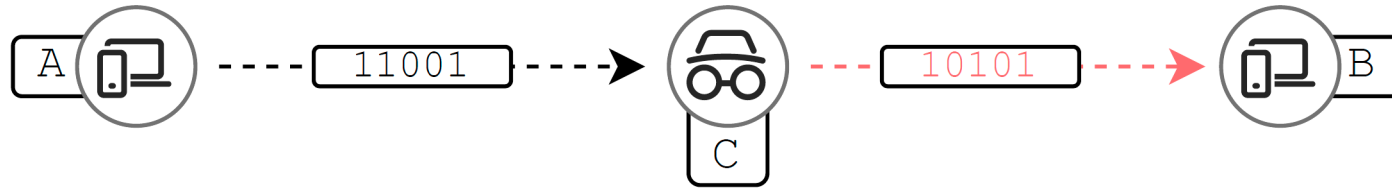
Denial of Service

- Annahme: Host A schickt Nachrichten an Host B. Ein Angreifer erlangt innerhalb des genutzten LANs Kontrolle über einen weiteren Host C , und gibt sich mittels ARP-Spoofing erfolgreich als Host B aus.
- Der Angreifer verwirft Nachrichten (ggf. selektiv) und blockiert Aktionen



Man-in-the-Middle

- Annahme: Host A schickt Nachrichten an Host B. Ein Angreifer erlangt innerhalb des genutzten LANs Kontrolle über einen weiteren Host C , und gibt sich mittels ARP-Spoofing erfolgreich als Host B aus.
- Der Angreifer **manipuliert Nachrichten** (ggf. selektiv) und leitet diese weiter



16.3 Sicherheitsmechanismen

Sicherheitsmechanismen

- Verschlüsselung (engl. encryption):
 - Daten sind nur für bestimmte Benutzer lesbar → Vertraulichkeit (confidentiality)
 - Prüfung auf unbefugte Änderungen → Integrität (integrity)
 - Grundlage für unabstreitbare Operationen
- Authentisierung (engl. authentication):
 - Überprüfung der Identität eines Benutzer, Client oder Servers, z.B. via Passwort
- Autorisierung (engl. authorization):
 - Überprüfung ob authentisierter Benutzer Zugriff hat.

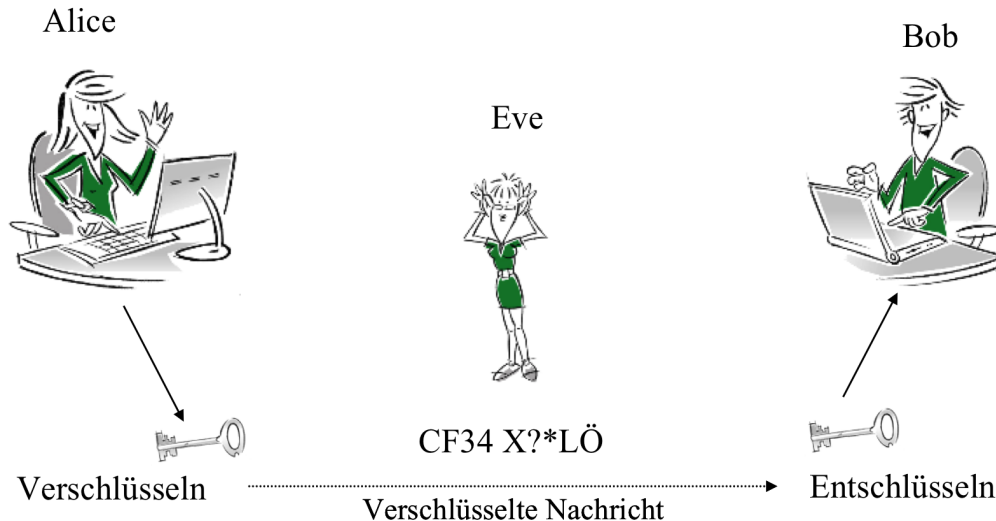
Sicherheitsmechanismen

- Auditing:
 - Speichern von Benutzeraktivitäten (z.B. unerlaubte Zugriffsversuche),
 - Für die spätere Analyse.
- Wichtig für Sicherheitspolitiken:
 - Zusammenspiel aller Mechanismen ist entscheidend.
 - Eine Schwachstelle bringt alle Maßnahmen zu Fall.

16.4 Symmetrische Verschlüsselung

Symmetrische Verschlüsselung

- Kommunikationspartner kennen beide einen geheimen Schlüssel (shared key)
 - Data Encryption Standard (DES), IBM, 1975 (56 Bit Key, veraltet)
 - Advanced Encryption Standard (AES), (128 – 256 Bit Key)
 - Für Kommunikation & Dateien sowie Authentisierung, ...

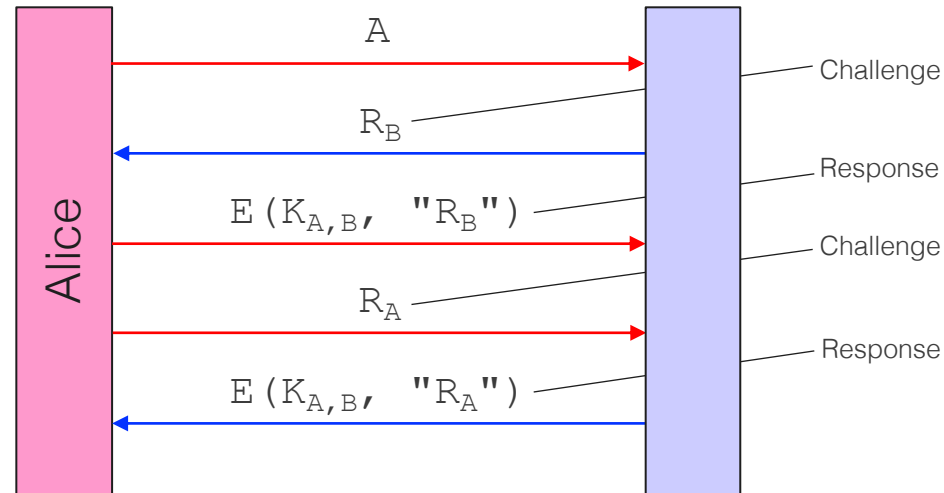


Symmetrische Verschlüsselung

- Funktionen:
 - Verschlüsselungsfunktion: E (encrypt): $E(K, T) \rightarrow C$
 - Entschlüsselungsfunktion: D (decrypt): $D(K, C) \rightarrow T$
 - K =Key , T =Text (zu verschlüsselnde Daten),
 $K_{A,B}$ geheimer Schlüssel, den nur A und B kennen
- Forderungen:
 - Falls $K_{A,B}$ unbekannt, soll es sehr aufwendig sein aus C das T zu ermitteln (Entschlüsselungsangriff).
 - Und es soll sehr aufwendig sein, aus T und C den Schlüssel $K_{A,B}$ zu ermitteln (Klartextangriff).

Challenge-Response-Protokoll

- Authentisierung zwischen zwei Servern
 - R_A und R_B sind Zufallszahlen
 - Nach Nachricht 3 ist A für B eindeutig identifiziert.
 - A muss noch sicherstellen, dass wirklich B auf der anderen Seite ist, dies erfolgt durch die Nachrichten 4 und 5



Challenge-Response-Protokoll

- Anschließender Datenaustausch über Verbindung:
 - Verschlüsselung der Daten mit $K_{A,B}$
 - Ausschluss von Verfälschung, sowie Einfügung und Verlust von Nachrichten.
- Wichtig: Beide Partner müssen $K_{A,B}$ kennen und sonst niemand.

Linux: gpg

message.json

Hexdump

```
00000000  7b 22 75 73 65 72 6e 61  6d 65 22 3a 22 68 65 69    |{"username":"hei|
00000010  6e 72 69 63 68 22 2c 22  70 61 73 73 77 6f 72 64    |nrich","password|
00000020  22 3a 22 68 65 69 6e 65  22 7d 0a                    |":"heine"}.|
```

Unter Linux kann das Tool `gpg` für die Verschlüsselung genutzt werden|



```
[student@uni]$ gpg --cipher-algo AES128 --batch --passphrase "abc123" -c message.json
```

- Verschlüsselter Text ist für einen Angreifer nutzlos

message.json.gpg

Hexdump

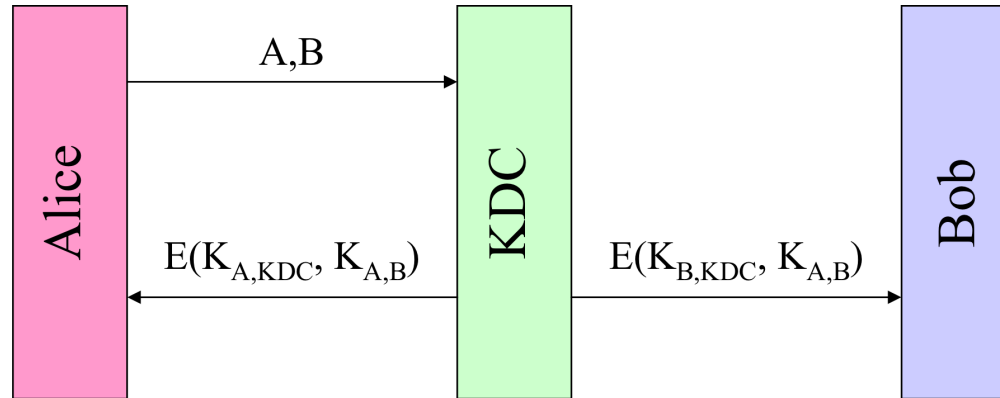
00000000	8c 0d 04 07 03 02 1c 40	79 03 77 99 e9 f4 f9 d2	@y.w.....
00000010	67 01 93 e4 09 67 c9 ad	f3 58 de 69 16 ee 82 0b	g....g...X.i....
00000020	b9 86 98 29 eb 31 2b a0	d4 8b d9 98 27 36 1f fe	...).1+.....'6..
00000030	10 ba fd 4f 47 0b f4 e0	2a e6 48 33 80 ff 35 4f	...0G...*.H3..50
00000040	82 dc 00 6c b2 51 63 32	54 b2 96 4c 84 43 a9 ee	...l.Qc2T..L.C..
00000050	62 28 35 c2 a4 c9 df 2d	b1 f7 09 d5 15 c3 40 3c	b(5....-.....@<
00000060	ec ed a4 cd d9 b1 f5 35	25 57 bc 81 15 67 9f e4	5%W...g...
00000070	e7 69 8a 9e 90 bd f9 2d		.i.....-

Schlüsselverwaltung

- Problem: Bei N Kommunikationspartnern sind $N \cdot (N-1)/2$ Schlüssel nötig
- Lösung: Key Distribution Center (KDC)
 - Alle Teilnehmer haben einen geheimen Schlüssel, den nur das KDC kennt
 - Somit sind bei N Kommunikationspartnern nur noch N Schlüssel notwendig
 - Für die Kommunikation zwischen zwei Knoten generiert das KDC einen Sitzungsschlüssel.

Schlüsselverwaltung

- Verbindungsabau zwischen A und B über KDC
 - KDC generiert Sitzungsschlüssel $K_{A,B}$ für einen Verbindungswunsch
 - $K_{A,B}$ wird durch KDC auf sicherem Weg (jeweils privater Key) zu A & B übertragen.
- Prinzip:



- Bem.: Sitzungsschlüssel wird nur für eine Sitzung verwendet und danach verworfen → schwerer zu knacken.

16.5 Asymmetrische Verschlüsselung

Asymmetrische Verfahren

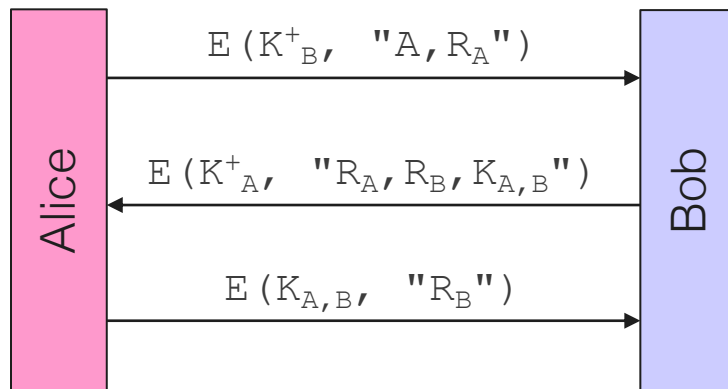
- Statt einem Schlüssel wird hier ein Schlüsselpaar pro teilnehmendem Computer verwendet
 - K^+ = öffentlich bekannter Schlüssel (engl. public key)
 - K^- = geheimer privater Schlüssel (engl. private key)
- Funktionen:
 - Kommunikation mit N Rechnern: $E(K^-, T') \rightarrow C'$ $D(K^+, C') \rightarrow T'$
 - Sender ist mithilfe von K^+ eindeutig identifizierbar
 - Kommunikation mit einem Rechner: $E(K^+, T) \rightarrow C$ $D(K^-, C) \rightarrow T$
 - Nur der Empfänger kann die Nachricht entschlüsseln
 - T und T' ist ein zu verschlüsselnder Text.

Asymmetrische Verfahren

- Forderungen an ein asymmetrisches Verfahren:
 - Wie bei symmetrischer Verschlüsselung,
 - Zusätzlich: aus bekanntem Schlüssel K^+ soll es nur mit hohem Aufwand möglich sein, den Schlüssel K^- zu berechnen
 - Zum Beispiel RSA
 - Benannt nach den Erfindern Rivest, Shamir und Adleman, 1977
- Vorteile asymmetrischer Verfahren:
 - Kein KDC erforderlich
 - Nur zwei Schlüssel für Kommunikation mit N Partnern
 - Aber Verbreitung des öffentlichen Schlüssels notwendig (siehe später)

Authentifizierung

- Anfrage kann nur von B entschlüsselt mit K_B^- werden.
- Antwort von B enthält Aufforderung R_B , damit A sich gegenüber B authentifiziert.
- Alice antwortet unter Verwendung des von B erzeugten Sitzungsschlüssels $K_{A,B}$ (nur sie kann diesen entschlüsseln)
- R_A und R_B sind Zufallszahlen, um das Einspielen alter Nachrichten zu erkennen.



16.6 Digitale Signatur

Hashfunktionen

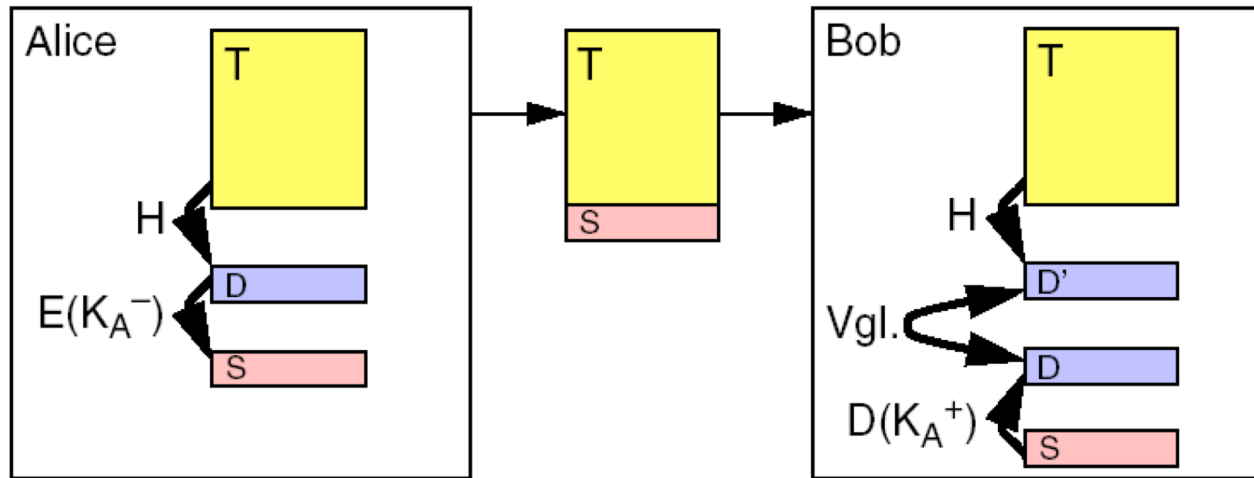
- Hashfunktion: $H(T) \rightarrow D$
 - T: Text
 - D: Hash-Wert fester Länge (Digest)
- Forderungen an eine sichere Hashfunktionen
 - H: einfach und schnell zu berechnen
 - **Einwegeigenschaft:** Für einen gegebenen Wert D ist es nahezu unmöglich ein T zu finden, so dass gilt: $H(T) = D$
 - **Kollisionsresistenz:** Wenn H bekannt ist, ist nahezu unmöglich ein Paar (T, S) zu finden, so dass gilt: $H(T) = H(S)$
 - Z.B. SHA-512 (kryptographische Hash-Funktion, Ausgabe 512 Bit)

Digitale Signatur

- Digitale Signaturen mit sicheren Hash-Funktionen:
 - Nachricht wird im Klartext versendet
 - Digest / Hashwert wird verschlüsselt angehängt = Signatur
 - Damit kann niemand den Hashwert manipulieren
- Nutzen
 - **Integrität:** Anhand der Signatur kann der Empfänger prüfen, ob die Nachricht verändert wurde
 - **Authentisierung:** Mithilfe der Signatur kann der Empfänger feststellen, wer die Nachricht gesendet hat

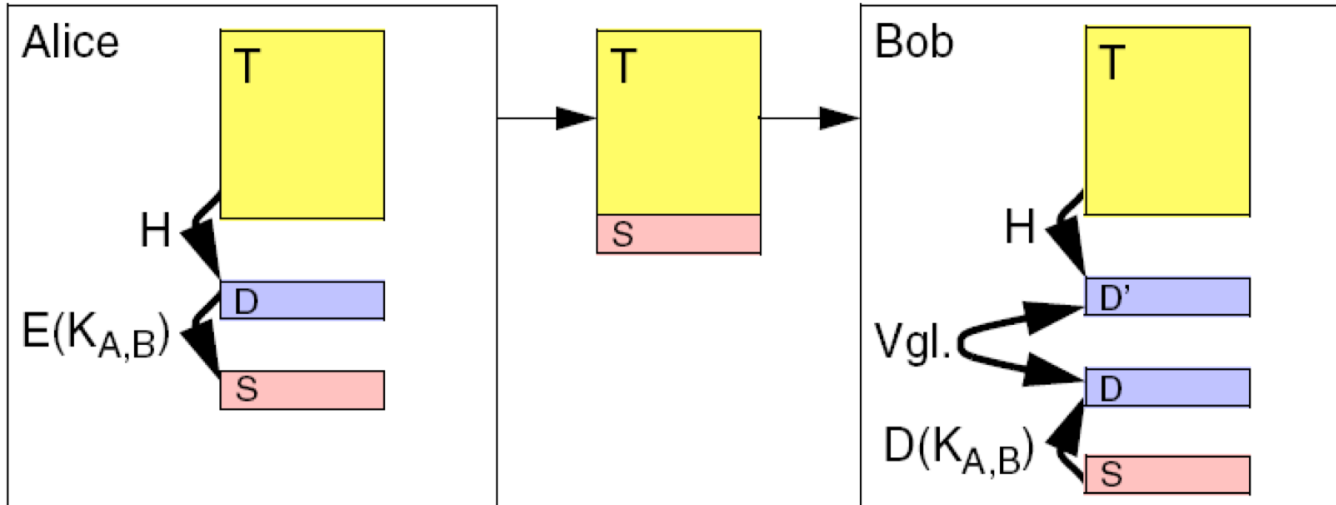
Digitale Signatur mit asymmetrischer Verschlüsselung

- Digitale Signatur mit asymmetrischen Verfahren:
 - Signatur durch Verschlüsselung des Hashwerts einer Nachricht mit dem privaten Schlüssel,
 - Öffentlicher Schlüssel dient zum Überprüfen der Signatur.



Digitale Signatur mit symmetrischer Verschlüsselung

- D = Digest; S = Signatur (verschlüsselter Digest)
- $K_{A,B}$ Schlüssel für Kommunikation zw. Alice und Bob



Schlüsselzertifizierung

- Problem: Vertrauen in den öffentlichen Schlüssel:
 - Öffentlicher Schlüssel enthält Besitzerangabe (z.B. E-mail-Adresse),
 - Wann kann man den Angaben trauen?
 - wenn sie von vertrauenswürdigen Subjekten bestätigt werden
- Lösung: Zertifikat = digital signierter öffentliche Schlüssel
 - Unterzeichnende Person bestätigt die Authentizität des öffentlichen Schlüssels

Zertifikatbeispiele

- Abstrakter Inhalt: Ich, der Aussteller, zertifiziere hiermit, dass ...
- ... das Fahrzeug mit der FIN WAZZGEXLAY28482 eine Betriebserlaubnis hat
- ... die Personalausweisnummer 3132552991 zu Max Mustermann gehört
- oder auch
- ... der Public Key X zum Host Y gehört
- Bem.: „zertifizieren“ bedeutet lediglich, dass der Aussteller etwas signiert/bestätigt

Schlüsselzertifizierung

- Beispiel: vertrauenswürdige Person Chuck signiert öffentlichen Key von Alice
 - Chuck zertifiziert den öffentlichen Schlüssel von Alice
 - Überzeugt sich, dass Alice privaten Schlüssel hat
 - Z.B. durch Challenge, Response und Verifizierung des Senders oder auf dem Postweg.
 - Bob vertraut Chuck:
 - Bob erhält öffentlichen Schlüssel von Alice,
 - Bob prüft diesen mithilfe des Zertifikates, also mithilfe der Signatur von Chuck (Integrität & Authentizität)
- Problem: Woher weiß Bob, dass Chucks öffentlicher Schlüssel zu Chuck gehört? → rekursives Problem.

Schlüsselzertifizierung - Lösung

- 1. Hierarchisch (Trust Center, z.B. Verisign):
 - Einer Wurzelautorität (Certification Authority) wird vertraut, deren K^+ ist wohlbekannt (Teil der Installation eines Betriebssystems)
 - Kette von Zertifikaten ergeben Vertrauensbasis.
- 2. Peer-to-Peer (z.B. PGP):
 - Mehrere digitale Signaturen für einen öffentlichen Schlüssel
 - Menge der vertrauenswürdigen Signaturen bestimmt letztlches Vertrauen

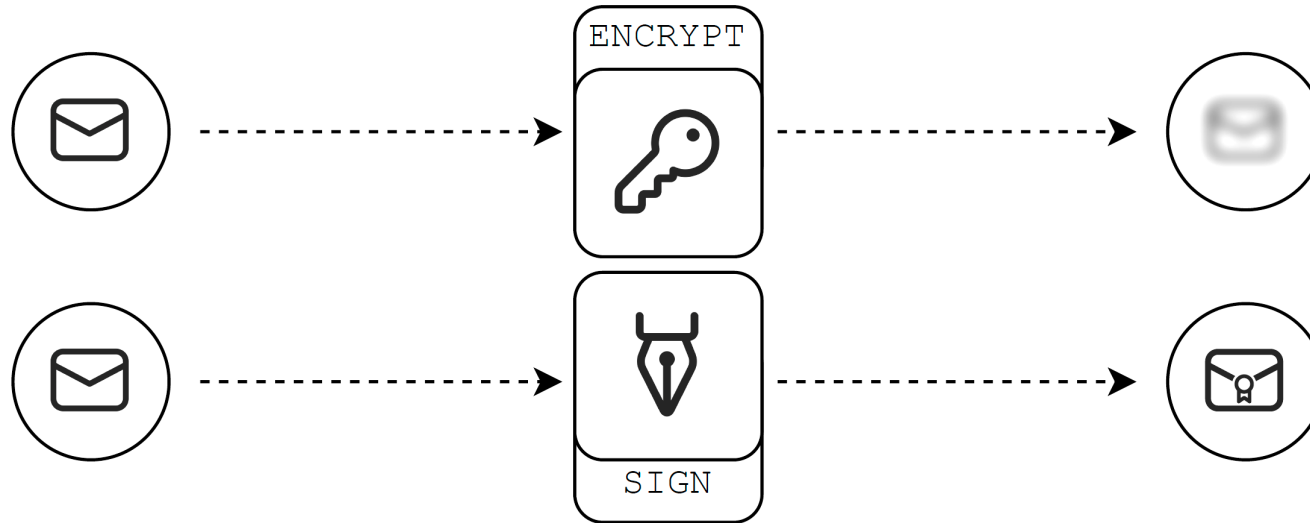
Schlüsselzertifizierung - In der Praxis

- Zertifikate nur für bestimmte Zeitdauer (Haltbarkeitsdatum)
- Zurückziehen von Zertifikaten:
 - Zertifizierungsstelle verwaltet Liste von zurückgezogenen Zertifikaten.
 - Empfänger muss die Signatur und die Liste prüfen.

16.7 Fallbeispiel: ARP Spoofing – Lösung

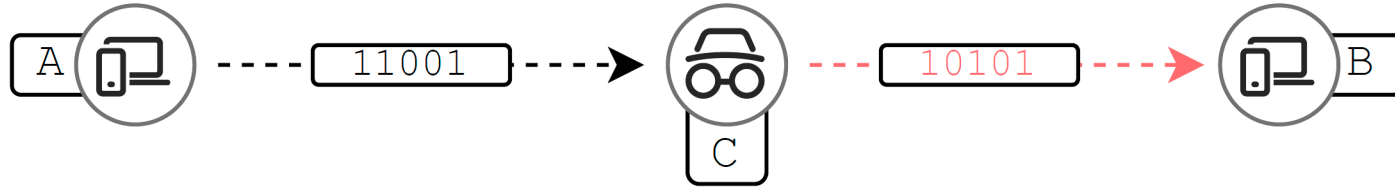
Schutzmaßnahmen

- Generell können zwei Maßnahmen gegen das (erfolgreiche) Abhören sowie die (erfolgreiche) Manipulation von Nachrichten ergriffen werden



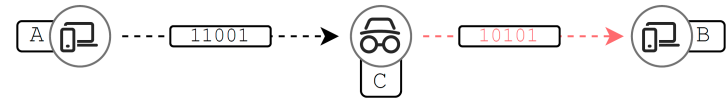
Man-in-the-Middle

- Auch wenn Daten verschlüsselt werden, kann ein Angreifer mittels ARP-Spoofing diese verändern ohne dass die beteiligten Parteien dies erkennen



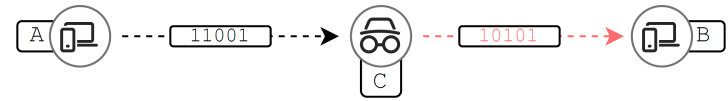
- Aus Sicht der beteiligten Parteien ist die andere Seite nicht erreichbar oder die ankommenden Nachrichten "kaputt"
- **Lösung:** Server weisen beim Verbindungsaufbau zunächst ihre Identität nach

Man-in-the-Middle Attack



- Was passiert, falls ein Angreifer auf einem dritten Host C während dem Schlüsselaustausch vorgibt das Ziel des Opfers zu sein (ARP-Spoofing)?
 1. Host A möchte mit Host B kommunizieren.
 2. Host C manipuliert mittels ARP-Spoofing die ARP-Tabellen von A und B:
 - A glaubt, Cs MAC-Adresse gehöre zu B.
 - B glaubt, Cs MAC-Adresse gehöre zu A.
 3. Gegenüber A gibt sich C als B aus und gegenüber B als A.
 4. C entschlüsselt Nachrichten von A, kann sie lesen oder verändern, und verschlüsselt sie anschließend erneut für B.
 5. A und B glauben jeweils, direkt miteinander zu kommunizieren.

Man-in-the-Middle Attack



- Durch den korrekten Einsatz von Zertifikaten, werden Man-in-the-Middle Angriffe praktisch unmöglich
- Beispiel: Host C gibt sich wieder als Host B aus:
 1. Host A versucht eine Verbindung mit Host B aufzubauen
 2. Host C fängt die Nachricht ab und gibt sich mithilfe eines gefälschten Zertifikates gegenüber A als Host B aus
 3. Host A prüft Zertifikat von C und erkennt dies sofort
- Hinweis: Host C kann weiterhin den Traffic umleiten, jedoch wird niemand mit ihm kommunizieren wollen, da sein Zertifikat ungültig ist

16.8 Vergleich der Verschlüsselungsverfahren

Vergleich

- Symmetrische Verschlüsselung
 - Für große Datenmengen, da weniger Rechenaufwand
 - Ein geheimer gemeinsamer Schlüssel $K_{A,B}$ für die Kommunikation zwischen zwei Personen
 - $K_{A,B}$ muss auf sicherem Weg ausgetauscht werden
- Asymmetrische Verschlüsselung
 - Für kleinere Datenmengen, da höherer Rechenaufwand
 - Ein Schlüsselpaar (Public-Key & Private-Key) pro Person
 - Kommunikation mit N Rechnern: $E(K^-, T') \rightarrow C'$ $D(K^+, C') \rightarrow T'$
 - Sender ist mithilfe von K^+ eindeutig identifizierbar
 - Kommunikation mit einem Rechner: $E(K^+, T) \rightarrow C$ $D(K^-, C) \rightarrow T$
 - Nur der Empfänger kann die Nachricht entschlüsseln

Vergleich

- Was passiert, falls ein Angreifer während dem Schlüsselaustausch mithört?
- Symmetrisch
 - Angreifer kann den $K_{A,B}$ abfangen und anschließend die gesamte Kommunikation mitschneiden und entschlüsseln
- Asymmetrisch
 - Angreifer kann lediglich den Public-Key “abfangen” und die Kommunikation im Anschluss nicht entschlüsseln, da er keinen Private-Key hat
- Lösung: $K_{A,B}$ wird mit Public Key des Empfängers verschlüsselt und in dieser Form versendet $\Rightarrow$ Angreifer “erkennt” nichts
 - Zusätzlich sollte $K_{A,B}$ periodisch neu erzeugt werden $\rightarrow$ „Sitzungsschlüssel“
 - Damit wird es schwieriger $K_{A,B}$ zu knacken

16.9 Zusammenfassung

Zusammenfassung

- In ungesicherten Netzwerken können Nachrichten abgehört, eingebracht und manipuliert werden
- Symmetrische Verfahren sind effizient → für große Datenmengen
- Asymmetrische Verfahren aufwändiger → für Authentisierung, Signaturen und Übertragung von Sitzungsschlüsseln
- Digitale Signaturen werden mit dem privaten Schlüssel erzeugt und mit dem zugehörigen öffentlichen Schlüssel überprüft.
- Zertifikate:
 - Bestätigen die Authentizität eines öffentlichen Schlüssels
 - Werden von einem Aussteller des Zertifikates digital signiert.
- Zertifikatsprüfung verhindert einen unbemerkten Man-in-the-Middle-Angriff